

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..25.3.4

FIREBASE

 Copy as Markdown

Firestore authentication

INFO

This page documents the legacy `serverpod_auth` module. To move an existing app to the current authentication framework, see [Migrate from legacy auth](#).

Serverpod uses [Firebase UI auth](#) to handle authentication through Firebase. It allows you to add social sign-in types that Serverpod doesn't directly support.

WARNING

Serverpod automatically merges accounts that are using the same email addresses, so make sure only to allow sign-ins where the email has been verified.

Server-side configuration

The server needs the service account credentials for access to your Firebase project. To create a new key go to the [Firebase console](#) then navigate to `project settings > service accounts` click on `Generate new private key` and then `Generate key`.



[Manage service account permissions](#)

🔑 **Firebase Admin SDK**

Legacy credentials

📁 **Database secrets**

All service accounts

🏠 **3 service accounts**

Firebase Admin SDK

Your Firebase service account can be used to authenticate multiple Firebase features, such as Database, Storage and Auth, programmatically via the unified Admin SDK. [Learn more](#)

Firebase service account
firebase-adminsdk-6ewzx@example-auth-f19cf.iam.gserviceaccount.com

Admin SDK configuration snippet

☒ Node.js ☐ Java ☐ Python ☐ Go

```
var admin = require("firebase-admin");  
  
var serviceAccount = require("path/to/serviceAccountKey.json");  
  
admin.initializeApp({  
  credential: admin.credential.cert(serviceAccount)  
});
```

Generate new private key

This will download the JSON file, rename it to `firebase_service_account_key.json` and place it in the `config` folder in your server project. Note that if this file is corrupt or if the name does not follow the convention here the authentication with firebase will fail.

Alternatively, you can provide the contents of the JSON file through the environment variable `SERVERPOD_PASSWORD_serverpod_auth_firebaseServiceAccountKey`.

⚠️ DANGER

The `firebase_service_account_key.json` file gives admin access to your Firebase project, never store it in version control.

Client-side configuration

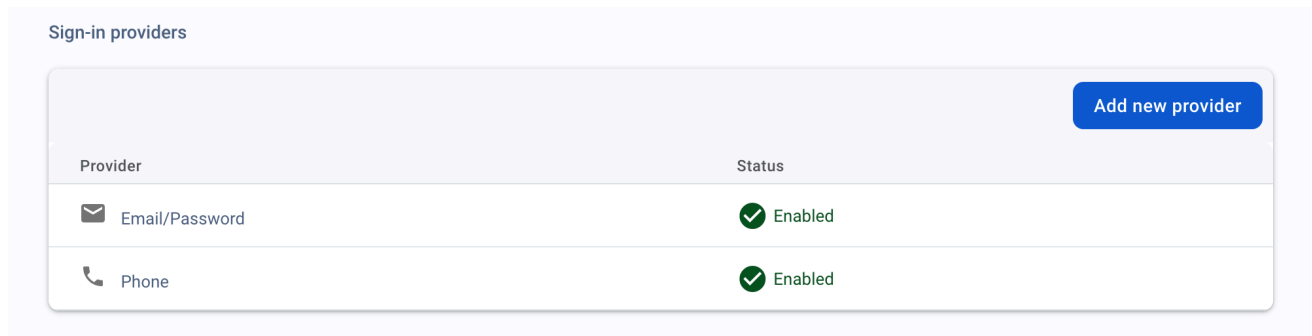
To add authentication with Firebase, you must first install and initialize the Firebase CLI tools and Flutter fire. Follow the instructions [here](#) for your Flutter project.

Firestore config

The short version:

```
$ flutter pub add firebase_core firebase_auth firebase_ui_auth
$ flutterfire configure
```

In the Firebase console, configure the different social sign-ins you plan to use, under **Authentication > Sign-in method**.



In your **main.dart** in your flutter project add:

```
import 'package:firebase_ui_auth/firebase_ui_auth.dart' as firebase;
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

...
void main() async {
  ...
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  firebase.FirebaseAuth.configureProviders([
    firebase.PhoneAuthProvider(),
  ]);

  ...
  runApp(const MyApp());
}
```

Trigger the auth UI with Serverpod

Add the `serverpod_auth_firebase_flutter` package.

```
$ flutter pub add serverpod_auth_firebase_flutter
```

The **SignInWithFirebaseButton** is a convenient button that triggers the sign-in flow and can be used like this:

```
SignInWithFirebaseButton(
  caller: client.modules.auth,
  authProviders: [
    firebase.PhoneAuthProvider(),
  ],
  onFailure: () => print('Failed to sign in with Firebase.'),
  onSignedIn: () => print('Signed in with Firebase.'),
)
```

Where **caller** is the Serverpod client you use to talk with the server and **authProviders** a list with the firebase auth providers you want to enable in the UI.

You can also trigger the Firebase auth UI by calling the method **signInWithFirebase** like so:

```
await signInWithFirebase(
  context: context,
  caller: client.modules.auth,
  authProviders: [
    firebase.PhoneAuthProvider(),
  ],
);
```

Where **context** is your **BuildContext**, **caller** and **authProviders** are the same as for the button. The method returns a nullable **UserInfo** object, if the object is null the Sign-in failed, if not the Sign-in was successful.